

Fixing Ruler-Like Border Artifact Bias in Vision Transformers for Skin Lesion Classification

Thanishk Palaparthi

1. Project Goal

The goal of this project is to design, train, and evaluate transformer-based models for multi-class skin lesion classification on the ISIC 2019 dataset, with a specific focus on reducing sensitivity to ruler-like border artifacts. In dermoscopic imaging, rulers, dark borders, and acquisition artifacts can become visual shortcuts that a model may learn instead of clinically meaningful lesion features. This project studies whether targeted border-artifact mitigation strategies can improve a scratch-trained transformer classifier.

The final task is an 8-class classification problem using the ISIC 2019 diagnostic labels MEL, NV, BCC, AK, BKL, DF, VASC, and SCC. The UNK class is excluded. The project compares a scratch-trained Vision Transformer baseline, a scratch-trained Swin Transformer proposed model, three Swin-based improvement techniques, and a zero-shot CLIP foundation model used only for inference.

The classification goal was achieved: the proposed Swin Transformer trained from scratch outperformed the baseline ViT on macro precision, macro recall, and macro F1. The best-performing improvement method was Technique 1, which combined border-focused preprocessing with synthetic ruler-like augmentation. The ruler-bias goal was addressed through artifact-mitigation strategies and qualitative visualization, but direct ruler robustness could not be measured because the dataset configuration used in this project does not provide explicit ruler/no-ruler annotations. Therefore, the ruler-related evaluation should be interpreted as a study of ruler-like border artifact mitigation rather than a definitive ruler-present versus ruler-absent robustness study.

2. Input and Ground Truth

The input to each trained model is a dermoscopic RGB skin-lesion image resized and normalized for transformer classification. The ground truth label is one of eight diagnostic classes from ISIC 2019.

Label	Class	Meaning
0	MEL	Melanoma
1	NV	Melanocytic nevus
2	BCC	Basal cell carcinoma
3	AK	Actinic keratosis

4	BKL	Benign keratosis
5	DF	Dermatofibroma
6	VASC	Vascular lesion
7	SCC	Squamous cell carcinoma

Table 1. ISIC 2019 diagnostic label mapping used in this project.

The model output is an 8-dimensional logit vector, where each logit corresponds to one diagnostic class, and the predicted class is the index with the highest softmax probability. The relationship between input and output is therefore standard supervised image classification: each image is mapped to one diagnosis label.

Rows marked as UNK are removed during dataset setup, and the remaining samples are required to have one positive label among the eight retained classes. Compared with the proposal-stage framing, the final input and ground-truth representation remained a supervised dermoscopic-image classification task. The main change is in the ruler-bias evaluation: the project originally targeted ruler-related bias, but the final dataset configuration did not include explicit ruler/no-ruler annotations. As a result, the ruler component is framed as ruler-like border-artifact mitigation using targeted interventions, robustness checks, and qualitative attention/error analysis rather than as a direct annotated ruler-bias benchmark.

3. Dataset Description

The dataset used in this project is the public ISIC 2019 Kaggle dataset of dermoscopic skin lesion images with diagnostic ground-truth labels and metadata. After excluding UNK, the final dataset contains 25,331 JPG dermoscopic images with CSV label and metadata files. The source used in this project is the public Kaggle dataset:

<https://www.kaggle.com/datasets/andrewmvd/isic-2019>

The data were prepared by removing UNK samples, encoding the eight retained classes, and creating stratified train/validation/test splits.

Full filtered class distribution:

Label	Class	Count in filtered ISIC 2019 dataset
0	MEL	4522
1	NV	12875
2	BCC	3323
3	AK	867
4	BKL	2624
5	DF	239

6	VASC	253
7	SCC	628

Table 2. Filtered ISIC 2019 class distribution after excluding UNK.

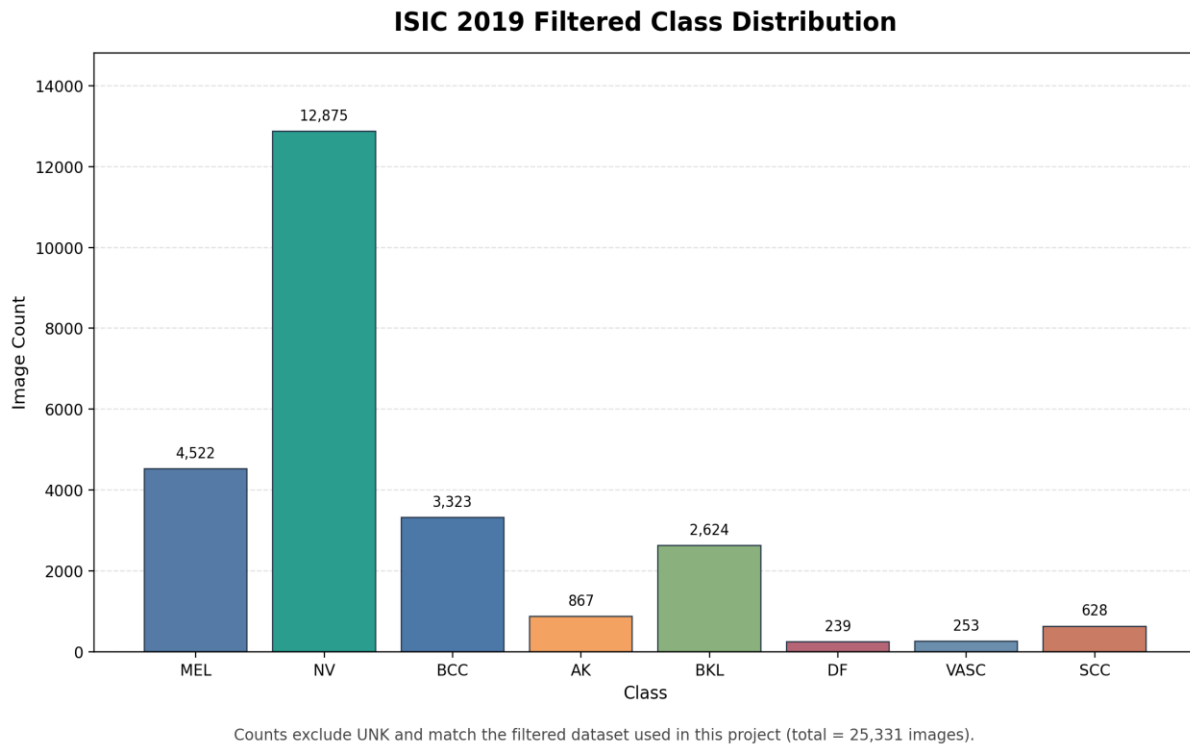


Figure 1. Filtered ISIC 2019 class distribution after excluding UNK.

3.1 Split sizes

The dataset was split into an 80/10/10 train/validation/test split using stratified sampling with seed 42.

Split	Count
Train	20264
Validation	2533
Test	2534
Test no-ruler proxy	2534
Test with-ruler proxy	2534

Table 3. Train/validation/test split sizes and proxy robustness split sizes.

The no-ruler and with-ruler proxy splits are duplicated from the same test split because explicit ruler/no-ruler annotations are not available in this dataset configuration. They are useful for checking evaluation consistency, but they should not be interpreted as evidence of real ruler-specific robustness.

3.2 Training split imbalance and class weights

The full filtered dataset is highly imbalanced, and the training split preserves that imbalance after stratification. NV dominates the dataset, while DF, VASC, SCC, and AK are much smaller. Because class weights are computed from the training split, the table below reports the training counts and resulting class weights used for weighted cross-entropy. Figure 1 visualizes this imbalance directly, showing the dominance of NV relative to minority classes such as DF, VASC, and SCC.

Label	Class	Train Count	Class Weight
0	MEL	3618	0.7001
1	NV	10300	0.2459
2	BCC	2658	0.9530
3	AK	694	3.6499
4	BKL	2099	1.2068
5	DF	191	13.2618
6	VASC	202	12.5396
7	SCC	502	5.0458

Table 4. Training-split class counts and weighted cross-entropy class weights.

4. Data Processing

Dataset setup performs the following steps:

- Loads the ISIC 2019 ground-truth CSV and image paths.
- Removes rows belonging to the UNK class.
- Keeps the eight diagnostic classes used in this project.
- Converts the one-hot class columns into integer labels.
- Creates stratified train, validation, and test splits with seed 42.
- Stores split assignments as CSV files for reproducible reuse.

For the trained ViT and Swin models, the training transform applies RandomHorizontalFlip(p=0.5), RandomVerticalFlip(p=0.5), RandomRotation(20), ColorJitter(0.2, 0.2, 0.2, 0.1), Resize((224, 224)), ToTensor(), and ImageNet normalization with mean [0.485, 0.456, 0.406] and std [0.229, 0.224, 0.225]. Validation and test use deterministic preprocessing: Resize(224), CenterCrop(224), ToTensor(), and the same ImageNet normalization. Class weights are computed from the training split and passed to cross-entropy loss to reduce the effect of class imbalance.

The foundation model uses CLIP-specific preprocessing: Resize(224), CenterCrop(224), ToTensor(), and CLIP normalization with mean [0.48145466, 0.4578275, 0.40821073] and std [0.26862954, 0.26130258, 0.27577711]. Data leakage is avoided by creating the train,

validation, and test splits before model training and by selecting the best checkpoint using validation macro recall, not test performance. The test set is used only for final evaluation. The duplicated no-ruler and with-ruler proxy splits are derived from the same test set and are not used for training.

5. Methodology

5.1 Baseline Model

The baseline model is a Vision Transformer using the "vit_base_patch16_224" architecture from timm. It is prebuilt but not pretrained:

```
timm.create_model("vit_base_patch16_224", pretrained=False, num_classes=8)
```

This satisfies the project requirement that proposed models be trained from scratch. The baseline is trained on the ISIC 2019 training split with an 8-class classification head. Its final test performance is macro precision 0.3849, macro recall 0.4867, and macro F1 0.4048. It provides a transformer-based reference point for comparison against the proposed Swin Transformer and improvement techniques.

5.2 Proposed Transformer Model Architecture

The proposed model is a Swin Transformer using "swin_tiny_patch4_window7_224" from timm, also with pretrained=False and num_classes=8.

Architecture Summary

The summaries below were generated from the actual PyTorch/timm model definitions used in this project and provide a report-friendly alternative to a TensorFlow-style model.summary() view.

Model	Input / patching	Core depth	Attention dimensions /	Parameters	Output
Baseline ViT-B/16	224 x 224 RGB; 16 x 16 patch embedding -> 196 image tokens + CLS	12 transformer blocks	embed dim 768; 12 heads; MLP 3072	85.80M	8 logits
Proposed Swin-Tiny	224 x 224 RGB; 4 x 4 patch embedding with patch merging	4 stages with depths [2, 2, 6, 2]	dims [96, 192, 384, 768]; heads [3, 6, 12, 24]; window size 7	27.53M	8 logits

Table 5. Model architecture summary for the baseline ViT and proposed Swin-Tiny.

Model-Summary Style Architecture Snapshot

Generated from the actual PyTorch/timm model definitions used in this project

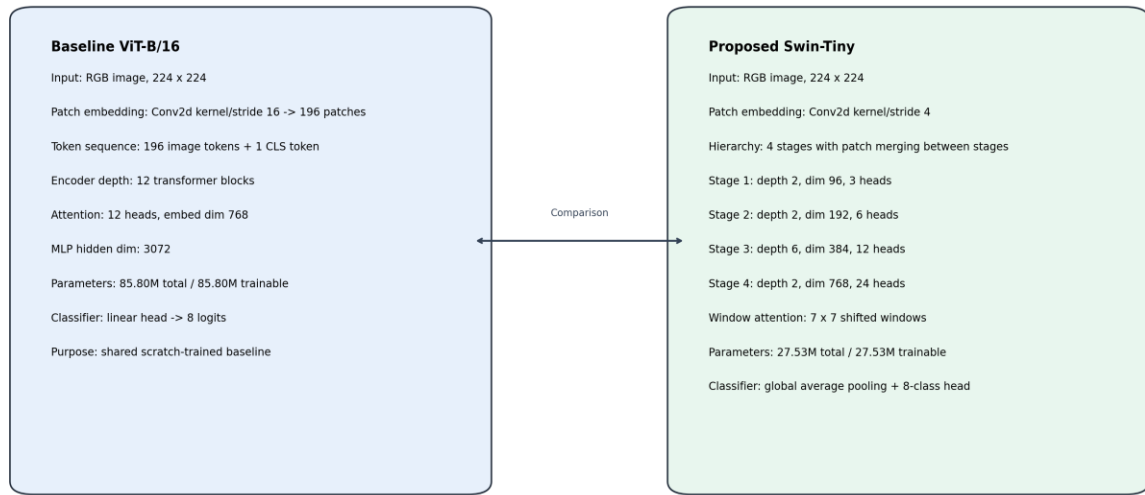


Figure 2. Model-summary style architecture snapshot for the scratch-trained baseline ViT and proposed Swin-Tiny.

Swin Transformer is appropriate for this problem because it uses hierarchical feature maps and shifted-window self-attention. Compared with a standard ViT, which tokenizes the whole image into fixed patches and applies global self-attention across a flat token sequence, Swin builds multi-stage local representations through windowed attention and shifted windows. This design reduces attention cost and encourages local-to-global feature learning, which is useful for dermoscopic images because diagnostic information may appear in local lesion texture, border irregularity, color variation, and spatial structure around the lesion.

The proposed Swin model is evaluated in four trained configurations:

- Swin without additional technique.
- Swin with Technique 1.
- Swin with Technique 2.
- Swin with Technique 3.

5.3 Training Details and Reproducibility

Training is implemented in PyTorch. The default training settings are:

Setting	Value
Framework	PyTorch
Model library	timm
Epochs	50
Batch size	64
Optimizer	AdamW

Learning rate	1e-4
Weight decay	1e-4
Scheduler	CosineAnnealingLR
Loss	Weighted cross-entropy
Seed	42
Checkpoint frequency	Every 10 epochs
Best checkpoint criterion	Validation macro recall
AMP	Optional, disabled by default

Table 6. Training configuration and reproducibility settings.

Best-model and periodic checkpoints were stored throughout training. Training can be resumed from an intermediate checkpoint, and each saved checkpoint stores enough information to restore the model, optimizer, scheduler, epoch, and best validation recall. The baseline and all Swin models are trained from scratch. No pretrained weights are used for these trained models. The foundation model is different by design: CLIP is pretrained, but it is used only for zero-shot inference and is not trained or fine-tuned in this project.

The experiments were run in a GPU-enabled Google Colab environment. During development, an NVIDIA A100 GPU on Google Colab was used for all the runs. The implementation also supports explicit control over device selection, batch size, worker count, mixed precision, save frequency, and checkpoint resumption.

Component	Environment used
Platform	Google Colab A100 runtime
Operating System	Linux
GPU	NVIDIA A100
GPU Memory	40 GB VRAM
System Memory	80 GB RAM
Dataset Placement	Dataset is stored in the Colab runtime environment (/dev/shm – System RAM) during training and evaluation.

Table 7. Hardware environment used for training and evaluation.

6. Performance Improvement Techniques

The project implements and evaluates three improvement techniques; all applied to the Swin Transformer.

Technique 1: Border/Ruler Preprocessing and Synthetic Ruler Augmentation

Technique 1 modifies the training transform by adding border-focused preprocessing and synthetic ruler-like augmentation. It applies RulerCropTransform and SyntheticRulerAugmentation before tensor conversion. The motivation is to reduce the model's ability to rely on border/ruler shortcuts and expose it to controlled ruler-like artifacts during training, encouraging the model to learn more lesion-relevant features instead of overfitting to acquisition artifacts. Technique 1 produced the best overall trained result, with the highest macro recall and macro F1 among all trained Swin runs.

Technique 2: Border-Patch Attention Regularization

Technique 2 adds an attention regularization loss with $\lambda_{\text{weight}}=0.1$. The regularizer penalizes attention assigned to predefined border patch indices. The motivation is to discourage the model from focusing too strongly on border regions, where ruler-like or image-frame artifacts are most likely to appear. This technique improved macro recall relative to Swin without techniques, but its macro F1 was lower than Technique 1.

Technique 3: Border-Patch Masking

Technique 3 uses a border patch masker with $\text{mask_prob}=0.5$. During training, border patch embeddings are masked so that the model cannot consistently depend on border information. The motivation is similar to dropout or input masking: by hiding potentially biased border features, the model may be forced to rely more on lesion-relevant image content. In practice, this was the weakest Swin configuration. It reduced accuracy and macro F1 relative to the Swin no-technique model, suggesting that the masking may have removed useful context or made optimization harder.

Technique Impact Summary

Run	Macro Precision	Macro Recall	Macro F1	Delta Recall vs Swin None	Delta F1 vs Swin None
Swin no technique	0.4048	0.5725	0.4342	0.0000	0.0000
Swin Technique 1	0.4104	0.5960	0.4471	+0.0234	+0.0129
Swin Technique 2	0.4003	0.5903	0.4215	+0.0178	-0.0127
Swin Technique 3	0.3731	0.5277	0.3850	-0.0448	-0.0492

Table 8. Performance impact of the three Swin improvement techniques relative to Swin without techniques.

Technique 1 is the only technique that improves both macro recall and macro F1 relative to Swin without techniques. Technique 2 improves macro recall but lowers macro F1, indicating that it likely increases sensitivity at the cost of precision. Technique 3 reduces both macro recall and macro F1, so it is not beneficial in its current configuration.

7. Performance Evaluation

The project evaluates models using macro precision, macro recall, macro F1, class-wise precision/recall/F1, confusion matrices, precision-recall curves, robustness comparison CSVs, and Grad-CAM visualizations. Macro-averaged metrics are important because the dataset is highly imbalanced: accuracy alone would be misleading because the dominant NV class is much larger than minority classes such as DF, VASC, and SCC.

Precision measures the fraction of predicted positives for a class that are correct. Recall measures the fraction of true samples for a class that are recovered by the model. F1 is the harmonic mean of precision and recall. Macro precision, macro recall, and macro F1 are computed by calculating the metric independently for each class and then averaging across the eight classes with equal weight. Accuracy is the fraction of all test samples classified correctly. The confusion matrices are row-normalized by true class, and the precision-recall curves are generated in a one-vs-rest format for each class. Grad-CAM is generated for the scratch-trained baseline and Swin models to support qualitative failure analysis; foundation Grad-CAM is skipped because zero-shot CLIP does not use the same classifier and Grad-CAM path as the trained project models.

7.1 Main Test Results

The table below reports the primary model-selection metrics: macro precision, macro recall, macro F1, and accuracy. Macro metrics remain the main emphasis because the dataset is strongly imbalanced, but accuracy is included for completeness on the shared test split.

Model / Run	Macro Precision	Macro Recall	Macro F1	Accuracy
Baseline ViT	0.3849	0.4867	0.4048	0.5770
Swin no technique	0.4048	0.5725	0.4342	0.5821
Swin Technique 1	0.4104	0.5960	0.4471	0.5797
Swin Technique 2	0.4003	0.5903	0.4215	0.5817
Swin Technique 3	0.3731	0.5277	0.3850	0.4925
Foundation CLIP zero-shot	0.1683	0.1430	0.1297	0.3662

Table 9. Main test-set comparison across baseline, Swin variants, and foundation model.

Swin improves over the ViT baseline. Technique 1 is the strongest trained configuration. Technique 2 trades F1 for recall. Technique 3 underperforms. The zero-shot CLIP foundation model is much weaker than trained models.

7.2 Swin Robustness Proxy Results

Run	Macro Recall	Macro F1	Accuracy
Swin no technique	0.5725	0.4342	0.5821
Swin Technique 1	0.5960	0.4471	0.5797
Swin Technique 2	0.5903	0.4215	0.5817
Swin Technique 3	0.5277	0.3850	0.4925

Table 10. Swin robustness proxy comparison on shared full-test / no-ruler / with-ruler splits.

Because the robustness_comparison.csv files contain identical Full Test, No Ruler, and With Ruler values for each Swin run, the table below reports the shared metric value once per run rather than repeating three identical columns. These results are interpreted only as consistency checks and not as a true ruler/no-ruler robustness experiment.

7.3 Graph and Visualization Analysis

Several graph-based artifacts support the quantitative evaluation: confusion matrices, precision-recall curves, epoch-trend plots, and Grad-CAM visualizations.

The confusion matrices highlight imbalanced performance and help identify which classes are commonly confused. The precision-recall curves show class separation across thresholds. Epoch-trend plots help verify training stability and checkpoint selection. Technique 3 has a logging gap for epochs 21–30 due to unrecoverable Colab logs, but its final checkpoints and evaluation artifacts were preserved. Grad-CAM visualizations for the baseline and Swin runs show whether attention concentrates on lesion regions, borders, or irrelevant artifacts, which is especially relevant for ruler-like artifact bias; however, they must be interpreted qualitatively because explicit ruler/no-ruler annotations are missing.

Confusion Matrices

These confusion matrices compare class-level error patterns for the baseline model, the strongest trained model, and the zero-shot foundation model. In practice, the best Swin model shows stronger diagonal concentration than the baseline across several classes, while the foundation model shows a much weaker and more diffuse error pattern consistent with its low macro metrics.

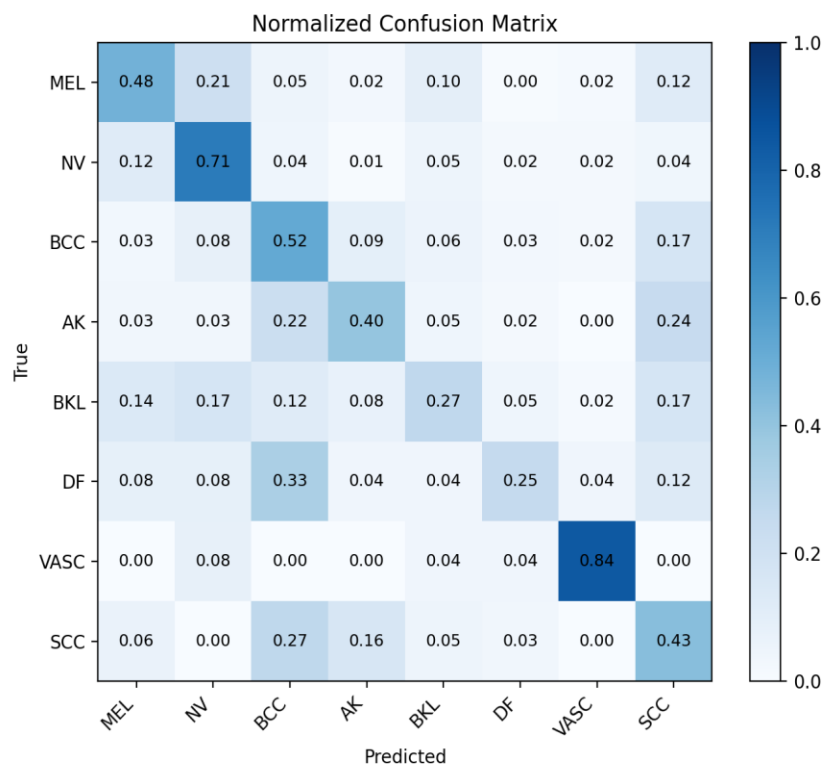


Figure 3. Baseline confusion matrix.

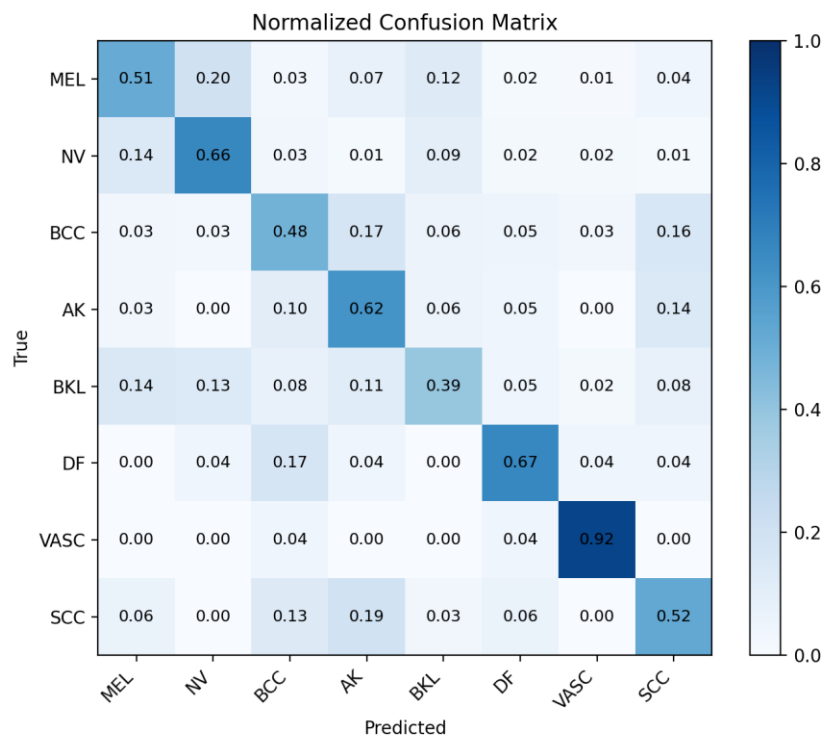


Figure 4. Best-model confusion matrix (Technique 1).

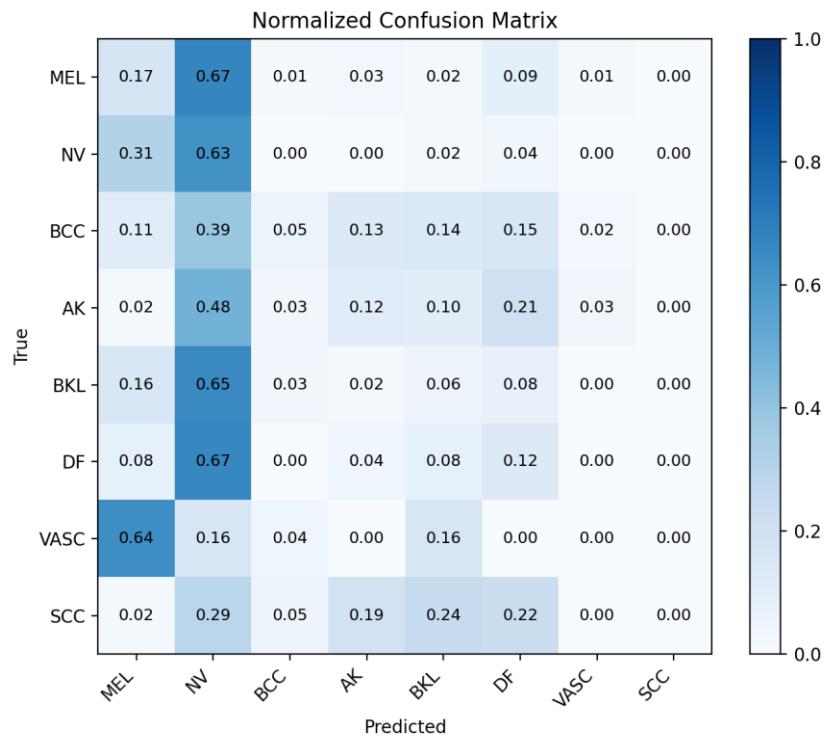


Figure 5. Foundation-model confusion matrix.

Precision-Recall Curves

These precision-recall curves show one-vs-rest class separation under class imbalance for the same three reference models. The trained baseline and Swin models produce more meaningful class-separation behavior than the zero-shot foundation model, and the strongest Swin run generally maintains the best balance between recall and precision across minority classes.

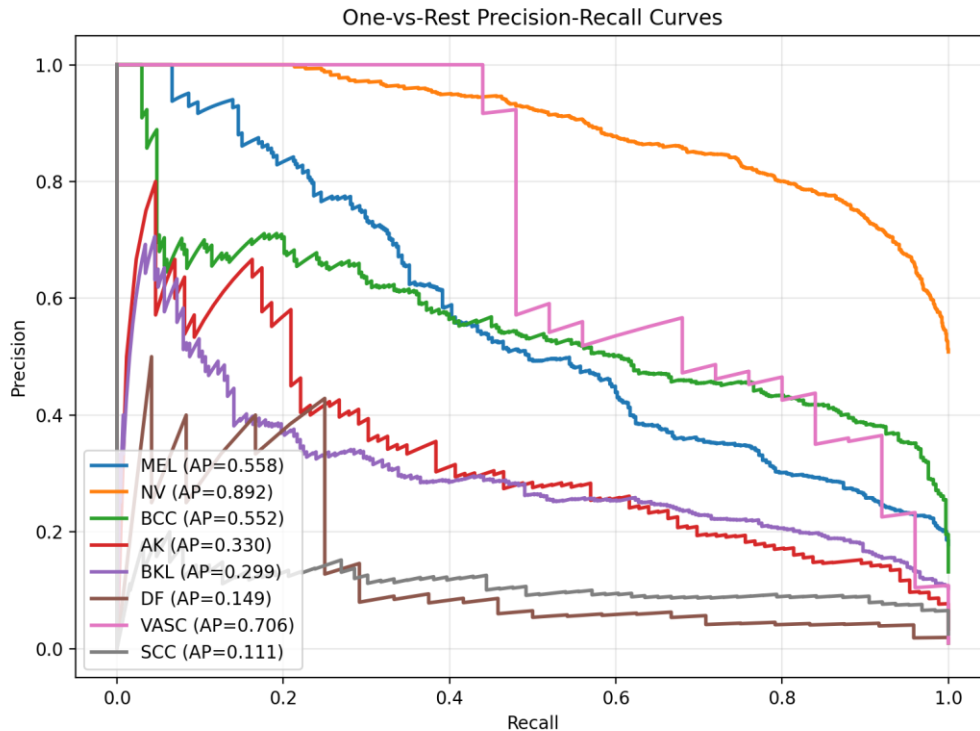


Figure 6. Baseline precision-recall curves.

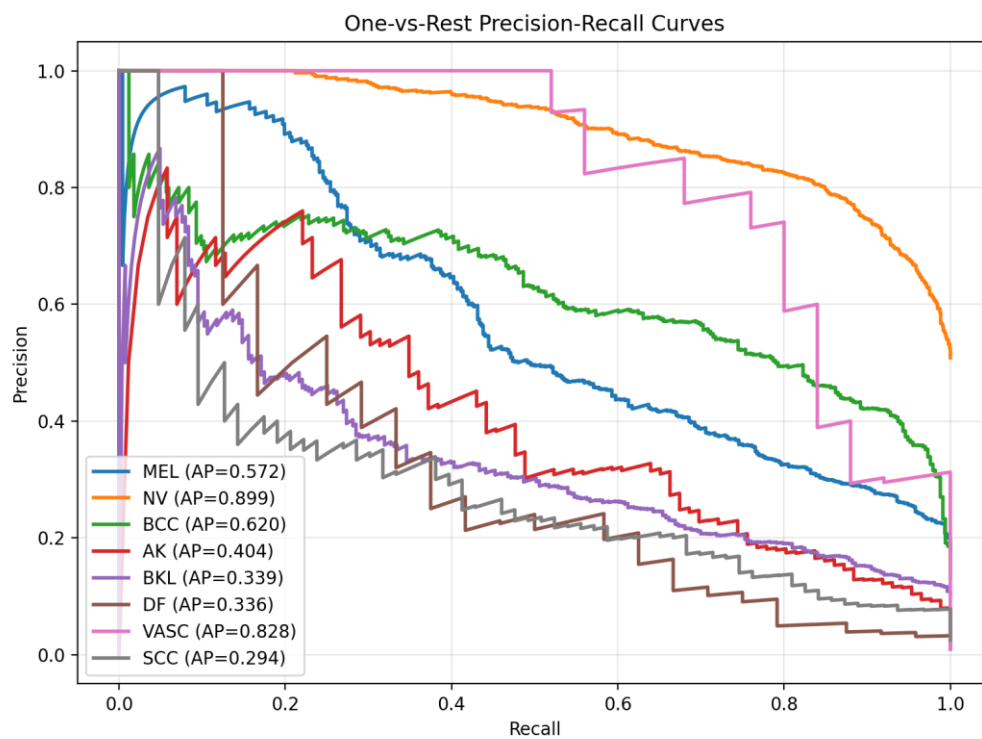


Figure 7. Best-model precision-recall curves (Technique 1).

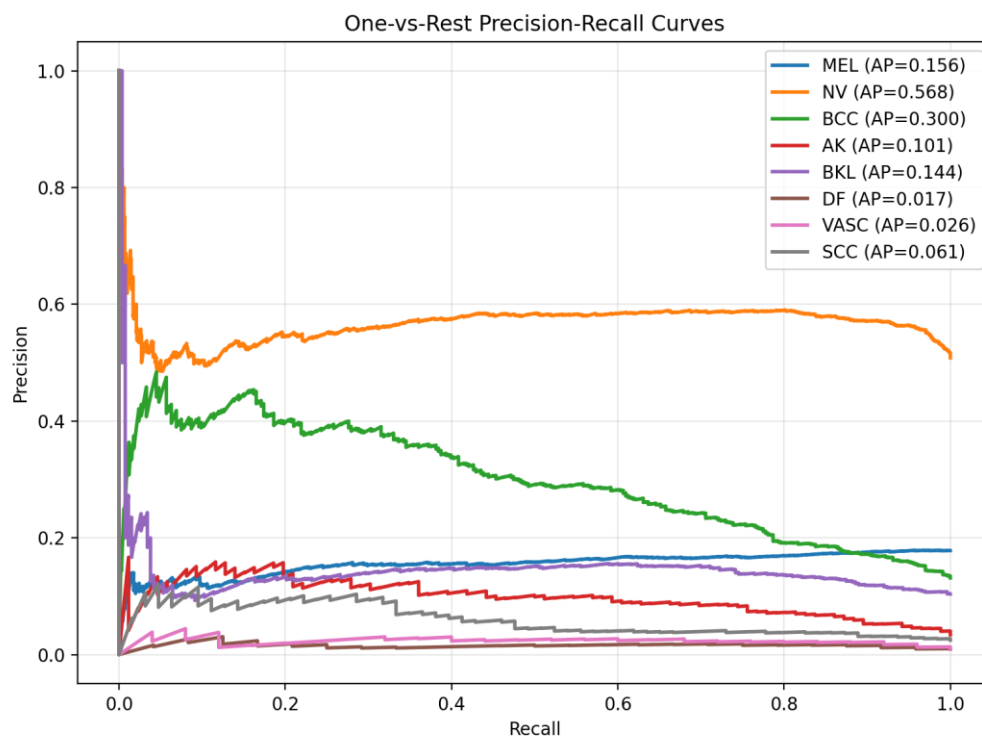


Figure 8. Foundation-model precision-recall curves.

Epoch Trends

The epoch-trend plots below show the training and validation behavior of the strongest trained run, Swin Technique 1.

Epoch 1	Train Loss: 2.1219	Val Loss: 1.9875	Val Recall (macro): 0.2197	Val F1 (macro): 0.1185
Epoch 2	Train Loss: 2.0057	Val Loss: 1.8548	Val Recall (macro): 0.2862	Val F1 (macro): 0.1592
Epoch 3	Train Loss: 1.9360	Val Loss: 1.8407	Val Recall (macro): 0.3138	Val F1 (macro): 0.2354
Epoch 4	Train Loss: 1.9080	Val Loss: 1.7765	Val Recall (macro): 0.3292	Val F1 (macro): 0.2272
Epoch 5	Train Loss: 1.8484	Val Loss: 1.7356	Val Recall (macro): 0.3338	Val F1 (macro): 0.2087
Epoch 6	Train Loss: 1.8178	Val Loss: 1.6951	Val Recall (macro): 0.3223	Val F1 (macro): 0.1991
Epoch 7	Train Loss: 1.7968	Val Loss: 1.7094	Val Recall (macro): 0.3449	Val F1 (macro): 0.2376
Epoch 8	Train Loss: 1.7517	Val Loss: 1.6585	Val Recall (macro): 0.3450	Val F1 (macro): 0.3058
Epoch 9	Train Loss: 1.7176	Val Loss: 1.5426	Val Recall (macro): 0.4164	Val F1 (macro): 0.2909
Epoch 10	Train Loss: 1.6745	Val Loss: 1.5710	Val Recall (macro): 0.3842	Val F1 (macro): 0.2561

Figure 9. Swin Technique 1 training and validation trends, epochs 1-10.

Epoch 11	Train Loss: 1.6439	Val Loss: 1.5442	Val Recall (macro): 0.3990	Val F1 (macro): 0.2725
Epoch 12	Train Loss: 1.6074	Val Loss: 1.5048	Val Recall (macro): 0.4270	Val F1 (macro): 0.3133
Epoch 13	Train Loss: 1.5600	Val Loss: 1.4764	Val Recall (macro): 0.4256	Val F1 (macro): 0.2939
Epoch 14	Train Loss: 1.5495	Val Loss: 1.4038	Val Recall (macro): 0.4633	Val F1 (macro): 0.3449
Epoch 15	Train Loss: 1.5133	Val Loss: 1.4654	Val Recall (macro): 0.4470	Val F1 (macro): 0.3146
Epoch 16	Train Loss: 1.4681	Val Loss: 1.4040	Val Recall (macro): 0.4561	Val F1 (macro): 0.3502
Epoch 17	Train Loss: 1.4443	Val Loss: 1.3868	Val Recall (macro): 0.4600	Val F1 (macro): 0.3310
Epoch 18	Train Loss: 1.4404	Val Loss: 1.3628	Val Recall (macro): 0.4826	Val F1 (macro): 0.3718
Epoch 19	Train Loss: 1.4341	Val Loss: 1.3903	Val Recall (macro): 0.4592	Val F1 (macro): 0.3390
Epoch 20	Train Loss: 1.3839	Val Loss: 1.3827	Val Recall (macro): 0.4523	Val F1 (macro): 0.3587
Epoch 21	Train Loss: 1.3919	Val Loss: 1.3420	Val Recall (macro): 0.4957	Val F1 (macro): 0.3749
Epoch 22	Train Loss: 1.3781	Val Loss: 1.3150	Val Recall (macro): 0.4956	Val F1 (macro): 0.3517
Epoch 23	Train Loss: 1.3709	Val Loss: 1.3577	Val Recall (macro): 0.4761	Val F1 (macro): 0.3792
Epoch 24	Train Loss: 1.3235	Val Loss: 1.3112	Val Recall (macro): 0.4814	Val F1 (macro): 0.3775
Epoch 25	Train Loss: 1.2976	Val Loss: 1.2880	Val Recall (macro): 0.4900	Val F1 (macro): 0.3808
Epoch 26	Train Loss: 1.3008	Val Loss: 1.3108	Val Recall (macro): 0.4777	Val F1 (macro): 0.3919
Epoch 27	Train Loss: 1.2746	Val Loss: 1.2843	Val Recall (macro): 0.5026	Val F1 (macro): 0.3836
Epoch 28	Train Loss: 1.2536	Val Loss: 1.2712	Val Recall (macro): 0.5138	Val F1 (macro): 0.3786
Epoch 29	Train Loss: 1.2422	Val Loss: 1.3341	Val Recall (macro): 0.4831	Val F1 (macro): 0.3527
Epoch 30	Train Loss: 1.2518	Val Loss: 1.2764	Val Recall (macro): 0.5131	Val F1 (macro): 0.4221
Epoch 31	Train Loss: 1.1990	Val Loss: 1.3143	Val Recall (macro): 0.5079	Val F1 (macro): 0.4072
Epoch 32	Train Loss: 1.1862	Val Loss: 1.2467	Val Recall (macro): 0.5227	Val F1 (macro): 0.4113
Epoch 33	Train Loss: 1.1680	Val Loss: 1.2418	Val Recall (macro): 0.5365	Val F1 (macro): 0.3917
Epoch 34	Train Loss: 1.1479	Val Loss: 1.2177	Val Recall (macro): 0.5426	Val F1 (macro): 0.4254
Epoch 35	Train Loss: 1.1276	Val Loss: 1.2190	Val Recall (macro): 0.5291	Val F1 (macro): 0.4121
Epoch 36	Train Loss: 1.1168	Val Loss: 1.2495	Val Recall (macro): 0.5374	Val F1 (macro): 0.4314
Epoch 37	Train Loss: 1.1021	Val Loss: 1.2330	Val Recall (macro): 0.5398	Val F1 (macro): 0.4078
Epoch 38	Train Loss: 1.0707	Val Loss: 1.2431	Val Recall (macro): 0.5442	Val F1 (macro): 0.4299
Epoch 39	Train Loss: 1.0772	Val Loss: 1.2208	Val Recall (macro): 0.5469	Val F1 (macro): 0.4208
Epoch 40	Train Loss: 1.0838	Val Loss: 1.2129	Val Recall (macro): 0.5508	Val F1 (macro): 0.4181
Epoch 41	Train Loss: 1.0477	Val Loss: 1.1980	Val Recall (macro): 0.5486	Val F1 (macro): 0.4154
Epoch 42	Train Loss: 1.0484	Val Loss: 1.1958	Val Recall (macro): 0.5637	Val F1 (macro): 0.4294
Epoch 43	Train Loss: 1.0332	Val Loss: 1.2074	Val Recall (macro): 0.5591	Val F1 (macro): 0.4447
Epoch 44	Train Loss: 1.0402	Val Loss: 1.2052	Val Recall (macro): 0.5645	Val F1 (macro): 0.4490
Epoch 45	Train Loss: 1.0345	Val Loss: 1.2028	Val Recall (macro): 0.5619	Val F1 (macro): 0.4380
Epoch 46	Train Loss: 1.0269	Val Loss: 1.2061	Val Recall (macro): 0.5645	Val F1 (macro): 0.4420
Epoch 47	Train Loss: 1.0196	Val Loss: 1.2034	Val Recall (macro): 0.5623	Val F1 (macro): 0.4425
Epoch 48	Train Loss: 1.0234	Val Loss: 1.1964	Val Recall (macro): 0.5664	Val F1 (macro): 0.4451
Epoch 49	Train Loss: 1.0304	Val Loss: 1.1963	Val Recall (macro): 0.5659	Val F1 (macro): 0.4444
Epoch 50	Train Loss: 1.0210	Val Loss: 1.1953	Val Recall (macro): 0.5667	Val F1 (macro): 0.4454

Figure 10. Swin Technique 1 training and validation trends, epochs 11-50.

8. Comparison of Approaches

The scratch-trained Swin Transformer without techniques improves over the scratch-trained ViT baseline. Baseline ViT achieved macro recall 0.4867 and macro F1 0.4048, while Swin without techniques achieved macro recall 0.5725 and macro F1 0.4342, suggesting Swin's hierarchical shifted-window design is better suited to this dataset.

Technique 1 is the best-performing trained approach, with macro recall 0.5960 and macro F1 0.4471, indicating that border/ruler-focused preprocessing and synthetic augmentation

improved the balance between detecting minority classes and maintaining precision. Technique 2 achieves macro recall 0.5903 but macro F1 0.4215, indicating recall gains at the expense of precision. Technique 3 underperforms all other Swin runs, with macro recall 0.5277, macro F1 0.3850, and accuracy 0.4925. The foundation CLIP model's macro F1 0.1297 is much worse than the trained models, as expected for zero-shot inference on a specialized medical task. Overall, the best model is Swin Technique 1 by macro recall and macro F1.

9. Foundation Model Comparison

The foundation model approach uses "openai/clip-vit-large-patch14" in zero-shot inference mode. CLIP is pretrained, but no project training or fine-tuning is performed; the model uses text prompts for each class and compares image embeddings against text embeddings. Prompts include: "a dermoscopic image of {}", "a photo of {}", and "a skin lesion showing {}", with class abbreviations expanded into names (e.g., MEL → melanoma, NV → melanocytic nevus, BCC → basal cell carcinoma).

Model	Macro Precision	Macro Recall	Macro F1
Foundation CLIP zero-shot	0.1683	0.1430	0.1297
Baseline ViT	0.3849	0.4867	0.4048
Best Swin Technique 1	0.4104	0.5960	0.4471

Table 11. Foundation-model comparison against the baseline ViT and best trained Swin model.

Likely reasons for CLIP's weaker performance include lack of ISIC-specific supervision, coarse prompts for subtle dermatological categories, visually similar classes requiring domain-specific supervision, dataset imbalance, and domain shift between CLIP's pretraining data and dermoscopic imaging. Quantitatively, the foundation model reaches macro precision 0.1683, macro recall 0.1430, and macro F1 0.1297, which is far below both the baseline ViT and the best trained Swin model. CLIP is a useful foundation comparison but not competitive with task-trained transformer models here.

10. Error and Failure Analysis

Class-wise metrics show the models struggle most with minority classes and clinically subtle categories (DF, SCC, AK, VASC), which have much smaller training counts than NV, MEL, and BCC. Even with class weighting, minority-class predictions remain unstable. Baseline ViT performs reasonably on large classes such as NV but has weaker macro performance because of minority-class failures. Swin improves macro recall substantially, indicating better sensitivity across classes, but also shows precision-recall tradeoffs. Some techniques increase recall by predicting minority classes more often, lowering precision.

- Technique 1 provides the best tradeoff, improving both macro recall and macro F1.
- Technique 2 improves recall but lowers F1, suggesting a higher false-positive burden.
- Technique 3 is clearly too aggressive, lowering accuracy and macro F1.

Model / Run	Failure Pattern	Evidence
Baseline ViT	Weak minority-class F1 despite reasonable NV performance	DF F1 = 0.1500 and SCC F1 = 0.1693, while NV F1 = 0.7685
Swin no technique	Better recall than baseline but still unstable minority-class precision	DF precision = 0.1628 and SCC precision = 0.1883
Swin Technique 1	Best overall tradeoff, but minority precision remains limited	Macro F1 = 0.4471; AK precision = 0.2637; SCC precision = 0.2129
Swin Technique 2	Recall-oriented behavior with lower F1 than Technique 1	VASC recall = 1.0000; VASC precision = 0.2427; macro F1 = 0.4215
Swin Technique 3	Border masking appears too aggressive	Accuracy = 0.4925; macro F1 = 0.3850
Foundation CLIP	Zero-shot model fails on several medical classes	VASC F1 = 0.0000 and SCC F1 = 0.0000

Table 12. Summary of model-specific failure patterns and supporting evidence.

These examples show that the strongest model is not simply the one with the highest recall on a single minority class. The best model must balance recall and precision across all eight classes; by that criterion, Swin Technique 1 is the strongest trained run. Grad-CAM visualizations for the baseline and Swin runs help show whether the trained models focus on lesion regions, borders, or unrelated artifacts. Because the project lacks explicit ruler/no-ruler annotations, the no-ruler and with-ruler comparisons are interpreted only as consistency checks, while the main bias-mitigation evidence comes from technique design, quantitative comparisons, confusion matrices, PR curves, and qualitative Grad-CAM analysis.

Grad-CAM Examples

The following Grad-CAM examples provide qualitative evidence about where the baseline and the best trained model focus for both correctly classified and misclassified cases. These examples support the error-analysis discussion by showing whether attention concentrates on clinically relevant lesion regions or drifts toward border context and other non-lesion structure.

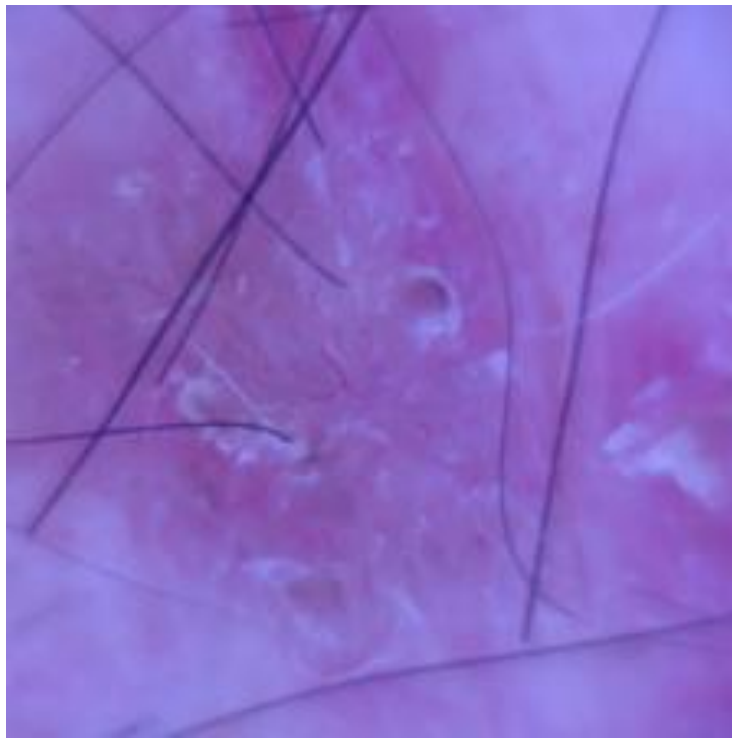


Figure 11. Baseline Grad-CAM example: misclassified case, true BCC, predicted SCC.

In Figure 11, the baseline misclassified BCC as SCC, illustrating that the baseline can fail even when the lesion belongs to a clinically important malignant class. This example is useful because it shows a failure case in which the baseline does not cleanly separate two visually similar diagnoses.

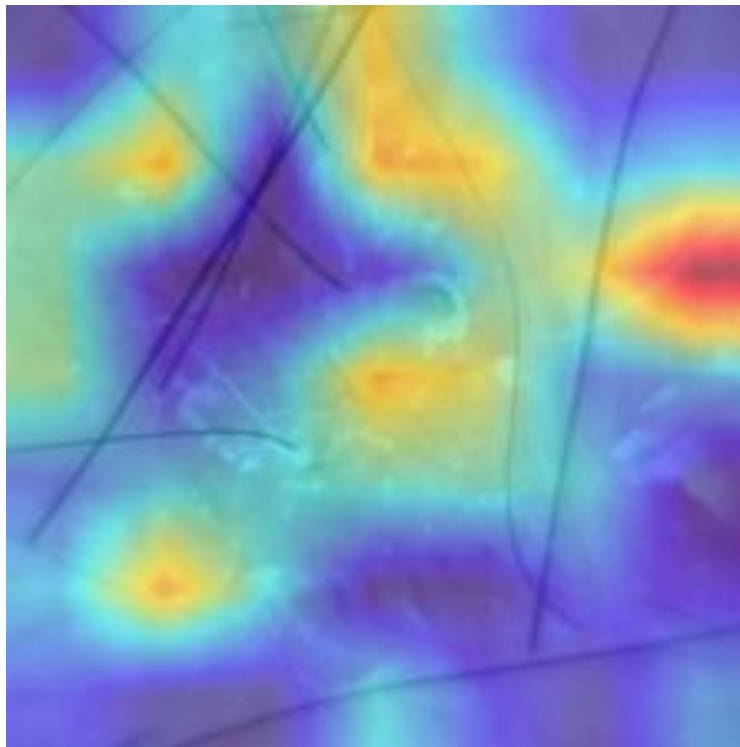


Figure 12. Technique 1 Grad-CAM example: correctly classified case, true BCC, predicted BCC.

In Figure 12, Technique 1 correctly classifies a BCC example, supporting the quantitative result that this configuration gives the best overall macro recall and macro F1. The example is consistent with the intended effect of border-focused preprocessing and synthetic ruler

augmentation: the model is encouraged to rely more on lesion-relevant image content than on border shortcuts.

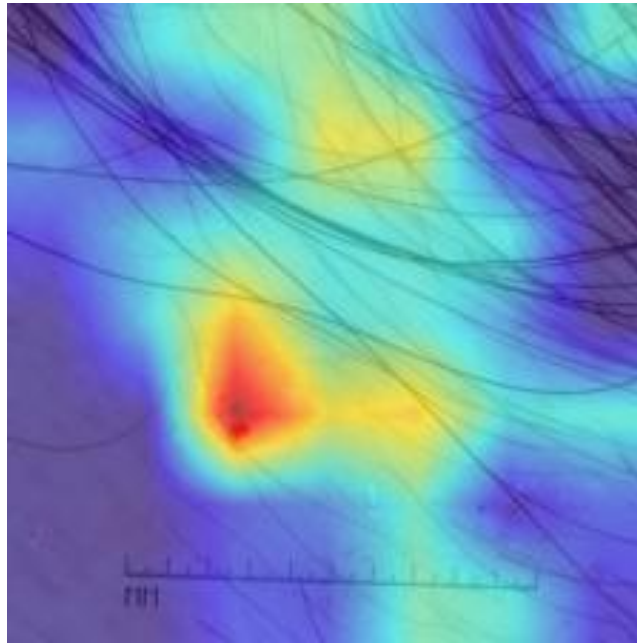


Figure 13. Technique 1 Grad-CAM example: misclassified case, true BKL, predicted BCC.

In Figure 13, Technique 1 still fails on a BKL-to-BCC confusion, showing that the strongest model is not error-free and that clinically subtle classes remain challenging. This qualitative example matches the class-wise metrics, where minority and visually similar classes still show limited precision despite the overall improvement in macro metrics.

11. Team Collaboration

The team used a shared problem definition, common dataset preparation, and consistent train/validation/test splits so that all transformer approaches were evaluated under the same protocol. All members used the same shared ISIC 2019 split, baseline model, and evaluation setup for fair comparison across architectures. Coordination was handled through weekly Microsoft Teams meetings, shared GitHub repositories for version control, and shared report artifacts so that experiments, metrics, plots, and final written results could be integrated consistently. Collectively, the team established the common dataset pipeline, the shared baseline and evaluation framework, the cross-model comparison structure, and the final report, presentation, and video deliverables.

12. Individual Contributions

The table below summarizes the primary architecture ownership for each member together with the main analysis and final-deliverable responsibilities that supported the combined project submission.

Team Member	Contributions
Jinali	Implemented and evaluated the Segmented ViT approach, analyzed its three improvement-technique variants, integrated the Segmented ViT results into the cross-model comparison, and contributed to the final report, presentation, and video deliverables.
Thanishk	Implemented and evaluated the Swin Transformer approach, including the three ruler/border-mitigation techniques used in this report, prepared the strongest trained-model analysis, and contributed to the shared experimental integration and final project deliverables.
Tommy	Implemented and evaluated the DeiT approach, analyzed its performance relative to the shared baseline, contributed DeiT results to the final comparison, and contributed to the final report, presentation, and video deliverables.

Table 13. Individual team-member contributions.

13. Conclusion

This project implemented and evaluated transformer-based skin lesion classifiers on ISIC 2019. The baseline ViT and proposed Swin Transformer were both trained from scratch, satisfying the no-pretrained-model requirement for project-trained models. The foundation model was used only as zero-shot CLIP inference and was not trained or fine-tuned.

The proposed Swin Transformer outperformed the baseline ViT. Among the three improvement techniques, Technique 1 achieved the best result with macro recall 0.5960 and macro F1 0.4471. Technique 2 improved recall but did not match Technique 1 in F1, while Technique 3 underperformed and likely masked too much useful image context. The foundation CLIP model performed substantially worse than the trained models, which is expected for zero-shot inference on a specialized medical-imaging task.

A key lesson learned is that bias-mitigation methods must be evaluated with both class-imbalance-aware metrics and dataset assumptions in mind: a technique can improve recall while still hurting overall F1, and a bias claim is weaker when explicit ruler annotations are unavailable. The main limitation is that the processed ISIC 2019 setup does not include explicit ruler/no-ruler annotations. Therefore, this project is best interpreted as a study of ruler-like border artifact mitigation rather than a definitive ruler-bias measurement study. Future work should use a curated ruler-annotated test set, tune the strength of border masking and attention regularization, and explore more clinically grounded prompts or fine-tuned medical foundation models. Despite this limitation, the project demonstrates a complete transformer classification pipeline with scratch-trained models, consistent splits, multiple improvement techniques, foundation-model comparison, quantitative metrics, and visual analysis.